



Postgrado en Ciencia de Datos e Inteligencia de Negocios:

Tópicos para la ciencia de datos y la toma de decisiones.

Presenta: Erick Varela Guzmán.

Contacto: evarela@uca.edu.sv

Universidad Centroamericana José Simeón Cañas.

Objetivos de esta sesión

- Programar un perceptrón simple utilizando una librería popular de Python.
- Comprender e introducir los conceptos fundamentales de las redes neuronales artificiales.
- Explorar la relación conceptual y funcional entre un perceptrón y una red neuronal multicapa.

2.1

Introducción



¿Que es Machine Learning?

Machine Learning (ML) o “aprendizaje automático”, puede definirse brevemente como una rama del campo de Inteligencia Artificial (IA), que permite a las computadoras hacer predicciones utilizando experiencias pasadas.



Estrategias de aprendizaje en ML



Aprendizaje supervisado

Se trabaja con datos etiquetados, para intentar encontrar una función que, dadas las variables de entrada, sea posible etiquetar un resultado de salida

Ejemplo: clasificación



Aprendizaje semisupervisado

En este modelo de trabajo, solo una fracción de los datos están etiquetados, con el objetivo de optimizar el proceso de aprendizaje

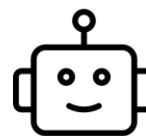
Ejemplo: medicina



Aprendizaje no supervisado

No se cuenta con datos etiquetados, por lo que el enfoque se encuentra en tratar de encontrar un criterio de organización de los datos.

Ejemplo: recomendaciones



Aprendizaje por refuerzo

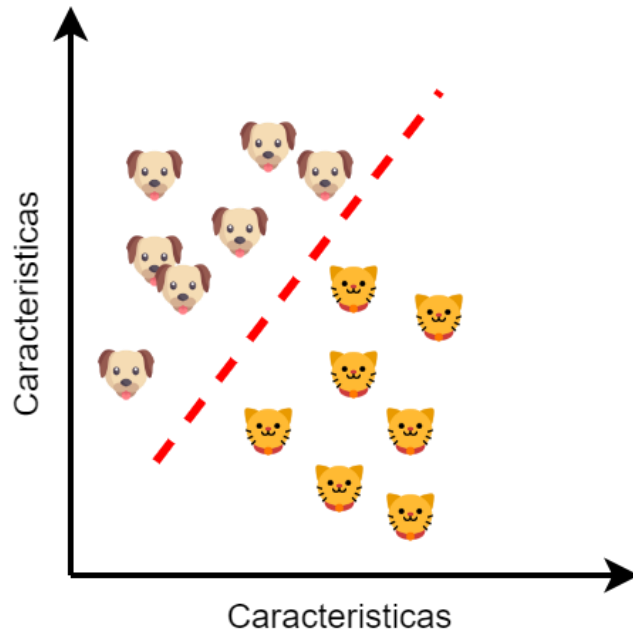
Modelo de trabajo que se basa en "recompensar" los comportamientos deseados y "penalizar" los no deseados.

Ejemplo: robots, videojuegos.

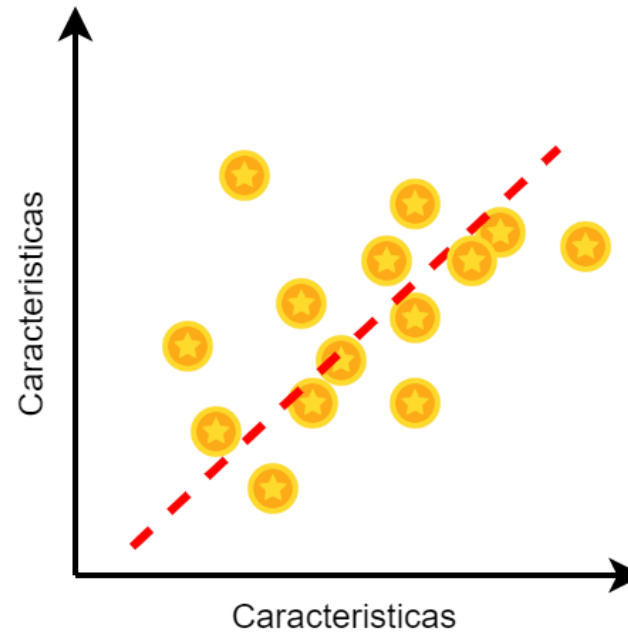
Algoritmos populares

	Supervisado	No supervisado
Categorico	Problemas de clasificación Algoritmos: • KNN • Arboles • Regresión logística • Naive-Bayes • SVM	Análisis de asociación Algoritmos: • Apriori • FP-Growth Modelo oculto de Márkov
Continuo	Problemas de regresión Algoritmos: • Regresión lineal • Regresión polinomial • Arboles de decisión • Random forests	Problemas de clustering Algoritmos: • SVD • PCA • K-means

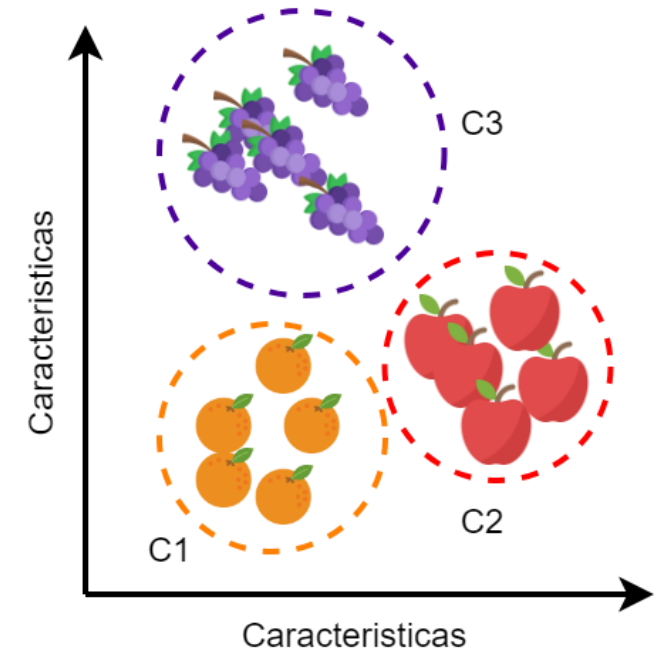
Problemas usuales y tratados en ML



**Problemas de
clasificación**



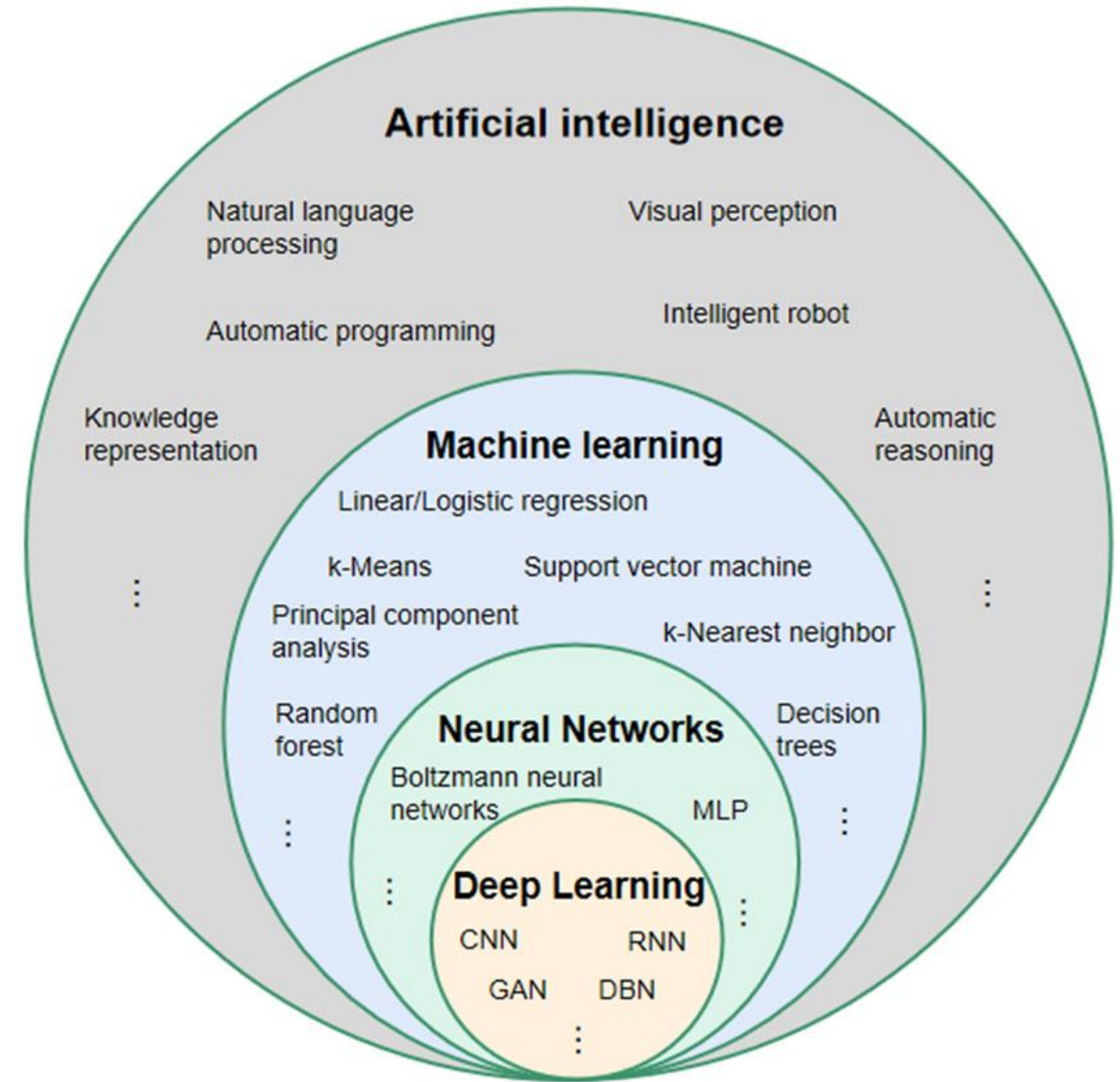
**Problemas de
regresión**



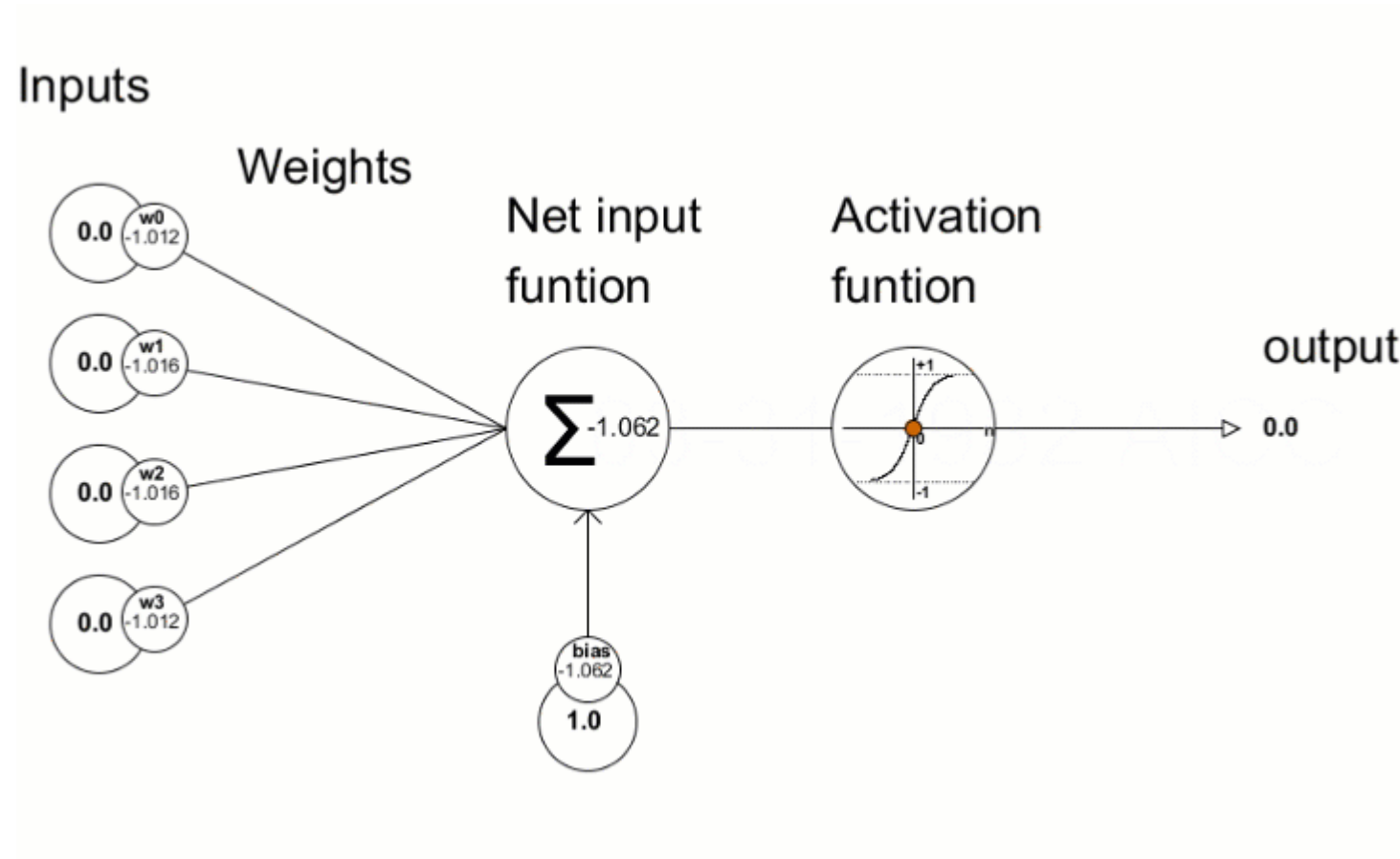
**Problemas de
agrupación
(clustering)**

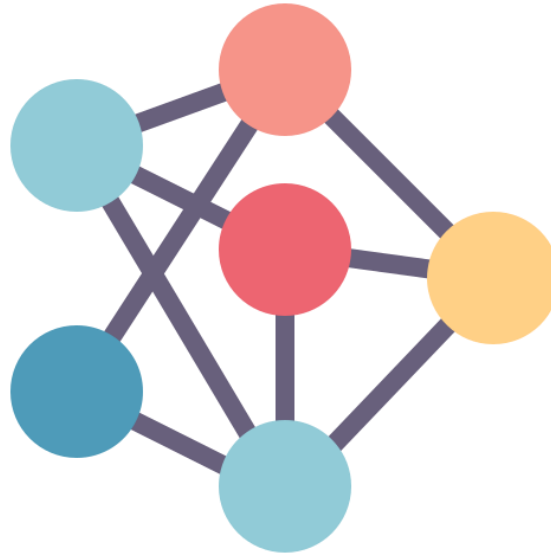
Redes neuronales

Las redes neuronales son un subconjunto del aprendizaje automático: son una forma de aplicar el aprendizaje automático.



Ejemplo de funcionamiento

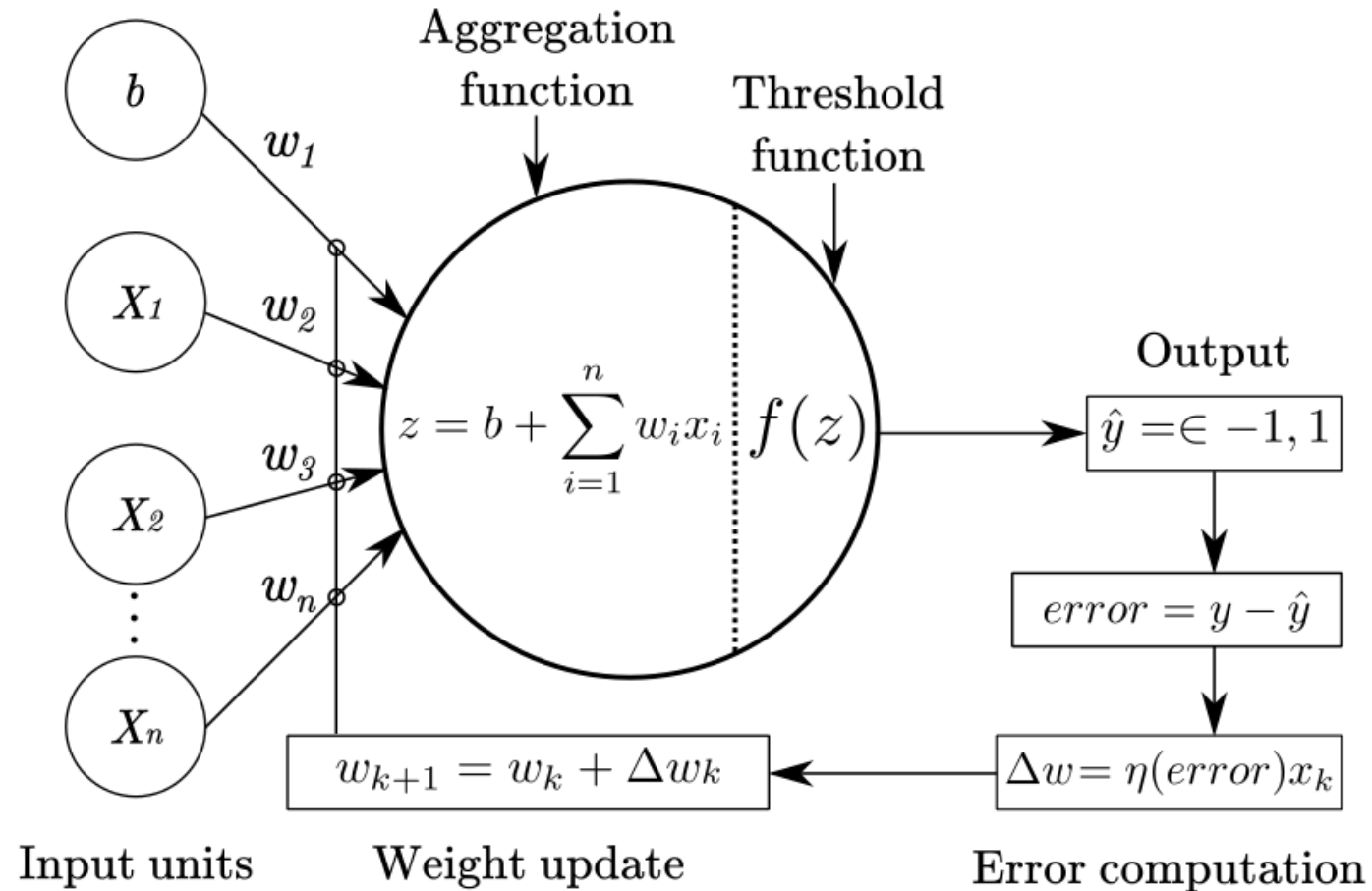




Ejemplos prácticos

Ejemplo 1. Programando un perceptrón simple utilizando en Python con la librería sklearn.

Modelo computacional



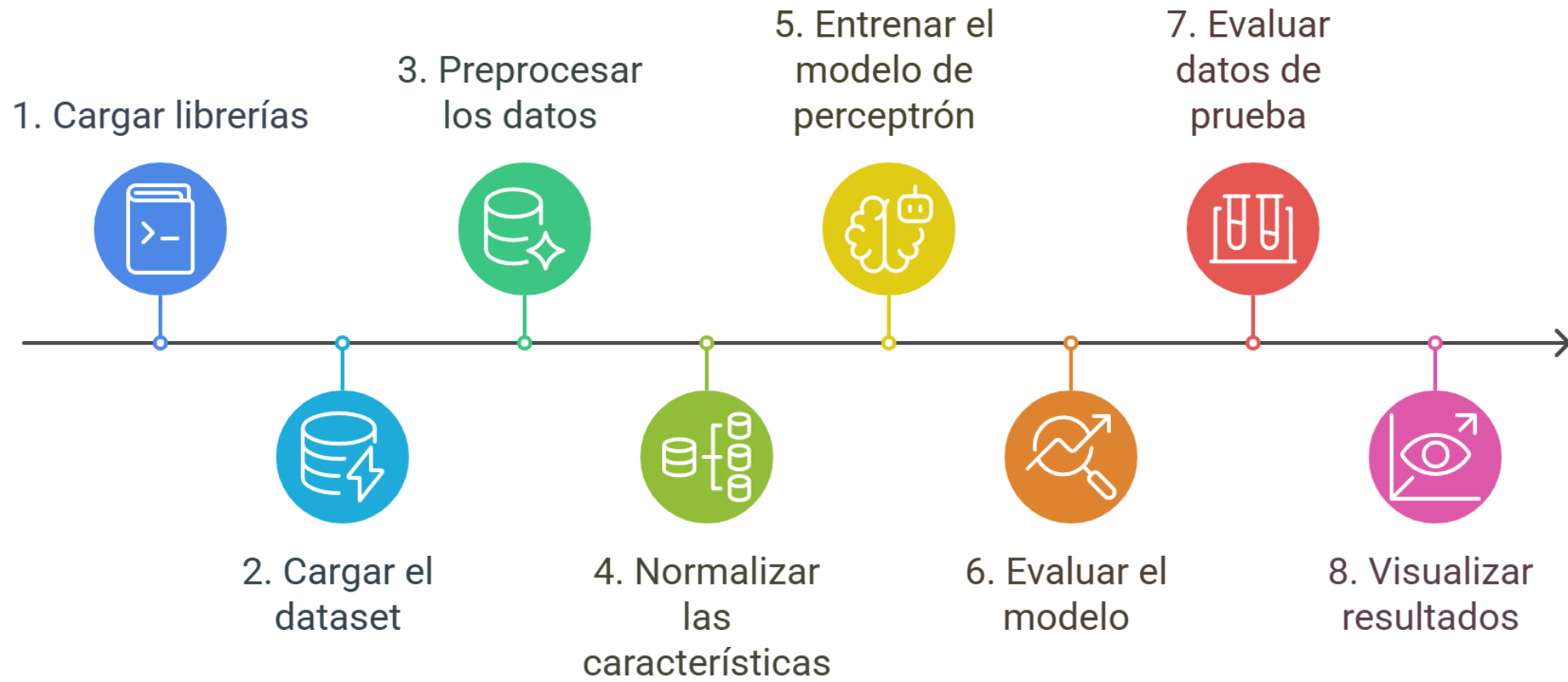
Librerías populares

Librería	Desarrollador	Dificultad	Dificultad	Enfoque
 TensorFlow	Google	Medio-Alto	Soporta redes profundas, producción, GPU/TPU, integración con Keras	Producción y proyectos avanzados
 Keras	Google (sobre TensorFlow)	Bajo	API amigable, ideal para prototipos rápidos	Principiantes y prototipado rápido
 PyTorch	Facebook (Meta)	Medio	Flexible, sintaxis clara	Investigación y educación

Librerías populares

Librería	Desarrollador	Dificultad	Dificultad	Enfoque
 Scikit-learn	Comunidad	Bajo	Enfoque en ML clásico, fácil de usar	Introducción al aprendizaje automático
 JAX	Google	Alto	Computación rápida, muy eficiente en GPU/TPU	Investigación y rendimiento avanzado
 Fastai	Comunidad (sobre PyTorch)	Bajo-Medio	Abstracción sobre PyTorch, buenas prácticas integradas, menos código	Enseñanza, prototipos rápidos y proyectos prácticos

Pasos para implementar un perceptrón simple



Conceptos importantes: división de datos

- En muchos casos basados en la implementación de métodos de Machine Learning, es usual utilizar una técnica basada en la división de datos para poder realizar la etapa de entrenamiento.
- Con SKLearn, la instrucción que facilita este procedimiento es **train_test_split()**. Esta instrucción se utiliza en el ejemplo práctico 1 de la siguiente manera:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Conceptos importantes: división de datos

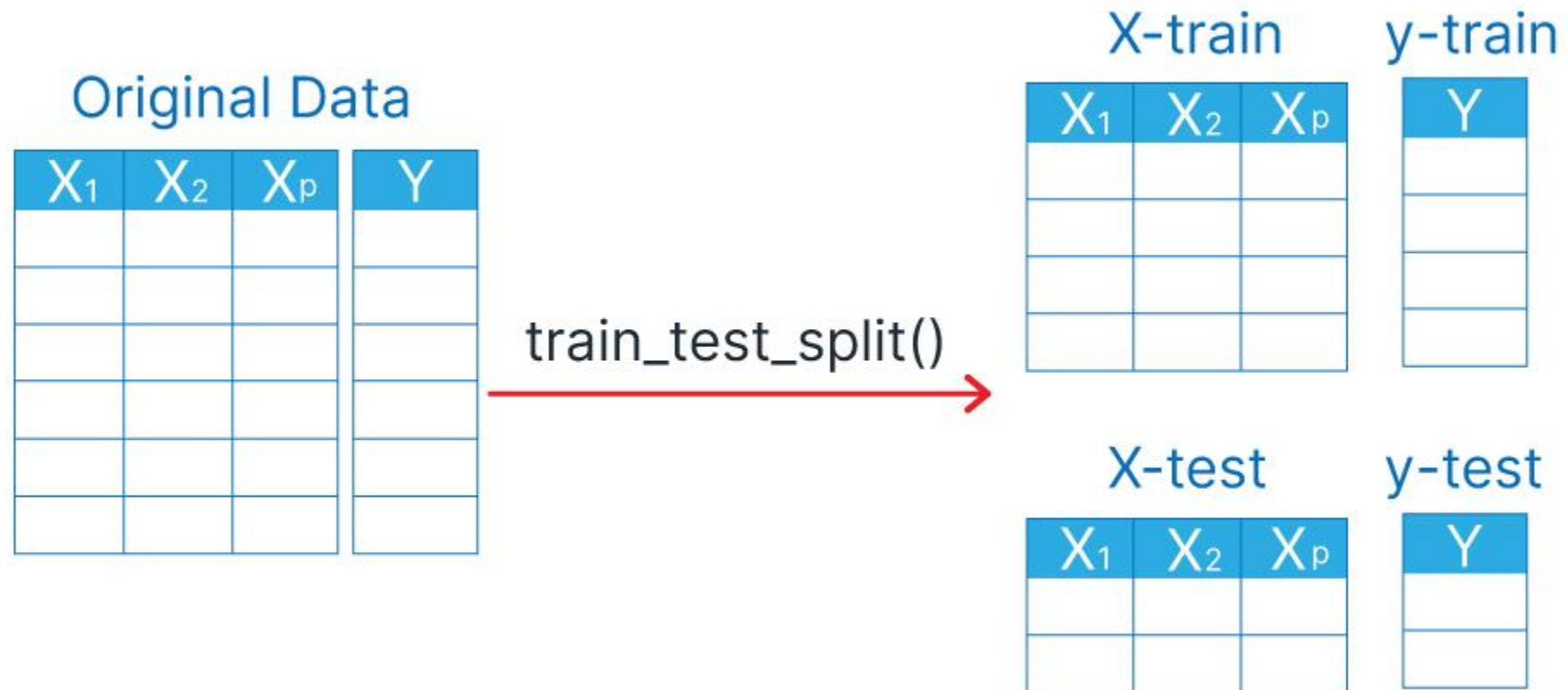
La instrucción `train_test_split()` divide los datos y los almacena en las siguientes 4 variables:

- **X_train**: Características de entrenamiento
- **X_test**: Características de prueba
- **y_train** : Etiquetas de entrenamiento
- **y_test** : Etiquetas de prueba

El parámetro "`test_size=0.2`" indica que el 20% de los datos se asignará al conjunto de prueba (`X_test`, `y_test`), mientras que el 80% restante se usará para entrenamiento (`X_train`, `y_train`).

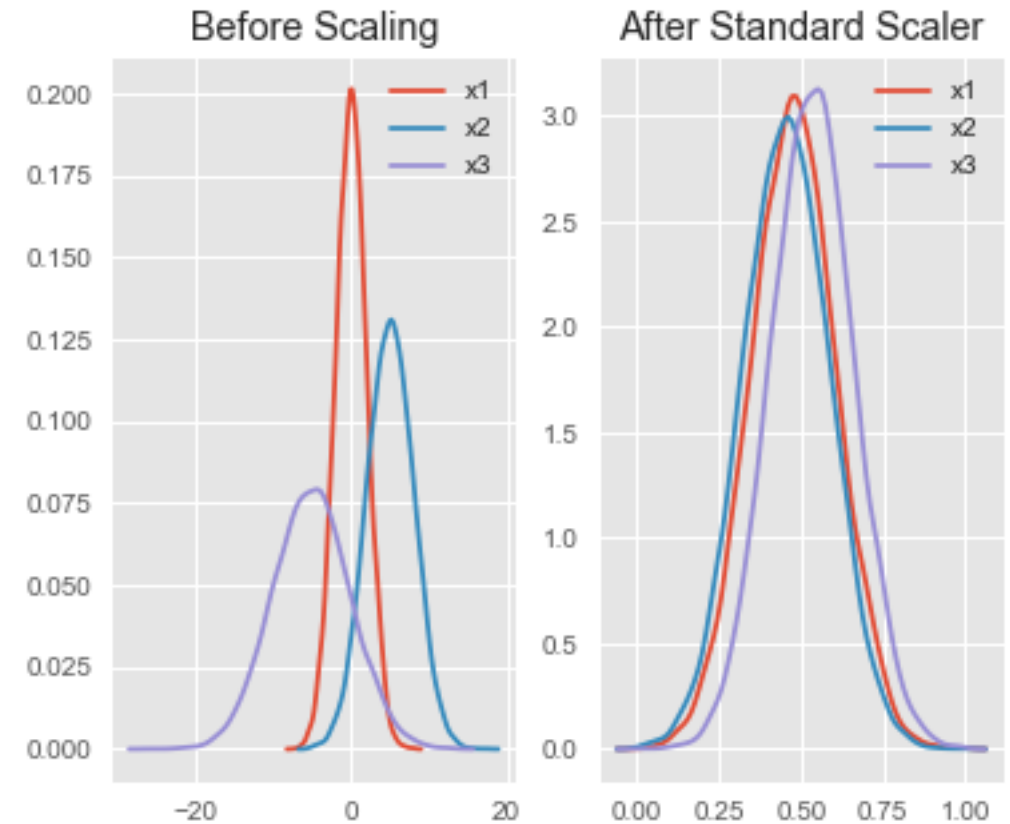
Conceptos importantes: división de datos

Gráficamente, la instrucción `train_test_split()` se puede comprender así:



Conceptos importantes: escalado de parámetros de entrada

- Algunos algoritmos de Machine Learning como las redes neuronales funcionan mejor cuando los datos tienen una escala similar, evitando que una característica domine a las demás.
- **StandardScaler** estandariza la escala de los datos. Lo que se busca es optimizar las condiciones para obtener mejores resultados en el proceso de entrenamiento.



Conceptos importantes: parámetros de entrenamiento

La creación de un perceptrón simple con SkLearn, necesita de algunos parámetros de configuración:

```
perceptron = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
```

- **max_iter:** Número máximo de iteraciones (épocas) durante el proceso de entrenamiento.
- **eta0:** Tasa de aprendizaje, controla el tamaño del ajuste en los pesos.
- **random_state:** Fija una semilla aleatoria para reproducibilidad de los resultados.

Conceptos importantes: evaluación del modelo

- La **precisión** es una de las métricas más comunes para evaluar modelos de clasificación porque da una idea de cuán preciso es el modelo al predecir las etiquetas correctas.
- En SkLearn, la instrucción `accuracy_score()` se utiliza para calcular la precisión de un modelo de clasificación. La precisión se calcula con la siguiente fórmula:

$$\text{Precisión} = \frac{\text{Número de predicciones correctas}}{\text{Número total de predicciones}}$$

- Es decir, se compara cuántas veces el modelo predijo correctamente y se divide por el total de ejemplos en el conjunto de prueba.

Instalando librerías necesarias

- La **precisión** es una de las métricas más comunes para evaluar modelos de clasificación porque da una idea de cuán preciso es el modelo al predecir las etiquetas correctas.
- En SkLearn, la instrucción `accuracy_score()` se utiliza para calcular la precisión de un modelo de clasificación. La precisión se calcula con la siguiente fórmula:

$$\text{Precisión} = \frac{\text{Número de predicciones correctas}}{\text{Número total de predicciones}}$$

- Es decir, se compara cuántas veces el modelo predijo correctamente y se divide por el total de ejemplos en el conjunto de prueba.



Sección de discusión y preguntas



Postgrado en Ciencia de Datos e Inteligencia de Negocios:

Tópicos para la ciencia de datos y la toma de decisiones.

Fin de la presentación